

AssetOpsBench: A Real-World Evaluation Benchmark for AI-Driven Task Automation in Industrial Asset Management

Dhaval Patel*
pateldha@us.ibm.com
IBM
Yorktown Heights, NY, USA

Nianjun Zhou*
jzhou@us.ibm.com
IBM
Yorktown Heights, NY, USA

Roman Vaculin
vaculin@us.ibm.com
IBM
Yorktown Heights, NY, USA

Shuxin Lin*
shuxin.lin@ibm.com
IBM
Yorktown Heights, NY, USA

Chathurangi Shyalika
jayakodc@email.sc.edu
University of South Carolina
Columbia, SC, USA

Natalia Martinez
Natalia.Martinez.Gil@ibm.com
IBM
Yorktown Heights, NY, USA

Jayant Kalagnanam
jayant@us.ibm.com
IBM
Yorktown Heights, NY, USA

James Rayfield*
jtray@us.ibm.com
IBM
Yorktown Heights, NY, USA

Suryanarayana R Yarrabothula
yarrabothula@iitbhillai.ac.in
IIT
Bhilai, Chathisgarh, India

Fearghal O'Donncha
feardonn@ie.ibm.com
IBM
Dublin, Ireland

Abstract

AI for Industrial Asset Lifecycle Management aims to automate complex operational workflows, such as condition monitoring and maintenance scheduling, to minimize system downtime. While traditional AI/ML approaches solve narrow tasks in isolation, Large Language Model (LLM) agents offer a next-generation opportunity for end-to-end automation. In this paper, we introduce AssetOpsBench, a unified framework for orchestrating and evaluating domain-specific agents for Industry 4.0. AssetOpsBench provides a multimodal ecosystem comprising a catalog of four domain-specific agents, a curated dataset of 140+ human-authored natural-language queries grounded in real industrial scenarios, and a simulated, CouchDB-backed IoT environment. We introduce an automated evaluation framework that uses three key metrics to analyze architectural trade-offs between the Agent-As-Tool and Plan-Execute paradigms, along with a systematic procedure for the automated discovery of emerging failure modes. The practical relevance of AssetOpsBench is demonstrated by its broad community adoption, with 350+ users and over 500 agents submitted to our public benchmarking platform, supporting reproducible and scalable research for real-world industrial operations.

 **Dataset**  **GitHub**  **Competition**  **Demo**

*These authors contributed equally and are considered co-primary authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3817562>

CCS Concepts

• Computing methodologies → Multi-agent planning.

Keywords

AI Agent, Multi-Agent System, LLM-As-Judge, AI Competition

ACM Reference Format:

Dhaval Patel, Shuxin Lin, James Rayfield, Nianjun Zhou, Chathurangi Shyalika, Suryanarayana R Yarrabothula, Roman Vaculin, Natalia Martinez, Fearghal O'Donncha, and Jayant Kalagnanam. 2026. AssetOpsBench: A Real-World Evaluation Benchmark for AI-Driven Task Automation in Industrial Asset Management. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770855.3817562>

1 Introduction

Industrial assets, such as data center chillers [33] and wind farms [31], are complex, multi-component systems that generate vast amounts of multimodal data, including time-series sensor readings, textual inspection and workorder records, operational logs, and images, throughout their lifecycle. The ability to monitor and interpret heterogeneous data from diverse sources, such as IoT SCADA [54] (Supervisory Control and Data Acquisition) sensors, operational KPIs, failure mode libraries, maintenance work orders, and technical manuals, is key to effective Asset Lifecycle Management [53]. However, subject matter experts such as maintenance engineers, site operators, and plant managers face considerable challenges in synthesizing insights from these disparate data streams to support timely and condition-aware decisions. As highlighted in Figure 1a,

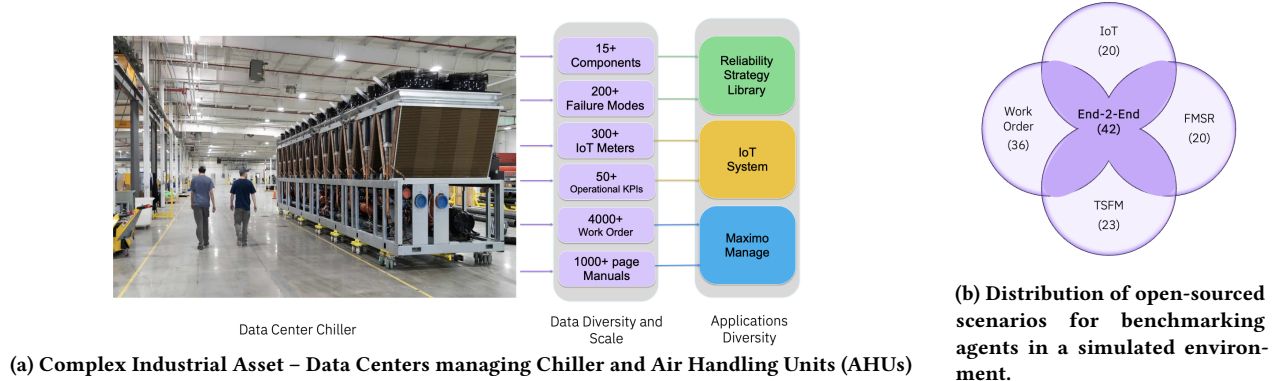


Figure 1: Industrial Asset Management Context and Utterance Scenario Distribution Used in AssetOpsBench.

the scale, semantic diversity of assets, and application-specific contexts often make traditional monitoring and management systems inadequate.

To address these challenges, the research and industrial communities are increasingly turning to AI agents: autonomous and goal-driven systems capable of integrating data, reasoning over complex conditions, and triggering appropriate actions. AI agents are particularly promising in the context of Industry 4.0, where the confluence of real-time IoT telemetry (e.g., Oracle IoT [35], enterprise asset management (EAM) systems [53], and IBM Maximo [19]) and reliability engineering frameworks necessitates scalable and intelligent automation. These agents promise to support a wide range of industrial workflows, from anomaly detection to maintenance scheduling, by bridging the gap between raw sensor data, maintenance report, work-order and business-level insights.

Despite significant advances in LLM-based agents and generalist models like ReAct [60], HuggingGPT [40], Chameleon [27], Magentic-One [10], and enterprise-ready computer use agents [29], a substantial gap remains in adapting these innovations for physical, sensor-driven industrial operations. While recent application-specific benchmarks such as ITBench [23], MLE-bench [4], τ -bench [59] and its multi-agent extensions [11], LangChain’s customer support benchmarks [46], TheAgentCompany [57], and CRMarena-Pro [15] offer rigorous evaluation in digital knowledge-work or IT domains, they do not address the unique challenges of industrial settings. These environments demand handling data modality diversity across time-series and text, managing complex business objects like asset hierarchies and failure modes, and facilitating collaboration across specialized operational personas.

Motivated by real-world deployment experience and the inherent application diversity of industrial environments (Figure 1a), we propose a **multi-agent architecture** that maps heterogeneous workflows to modular agents. In this architecture, specialized entities, such as an IoT agent for sensor telemetry and an FMSR agent for fault history, must intelligently collaborate to resolve complex queries like *“Why is the chiller efficiency dropping?”*, which require a deep integration of physical reasoning and operational semantics. To formalize the evaluation of such systems and bridge the gap between synthetic benchmarks and industrial reality, we introduce a unified framework ASSETOPS BENCH.

The key innovation and contributions in ASSETOPS BENCH are structured around three synergistic pillars:

- ♦ **Pillar I: The AssetOps Ecosystem.** We provide a high fidelity industrial sandbox featuring (1) a **catalog of specialized agents** (IoT, FMSR, TSFM, WO) with domain-specific tools; (2) **141 intent-aware scenarios** grounded in real data center assets (4 Chillers and 2 AHUs); and (3) a **dockerized environment with CouchDB access** for reproducible, end-to-end multi-agent testing.
- ♦ **Pillar II: Empirical & Diagnostic Framework.** We establish a rigorous methodology for agent evaluation, including a **comparative architectural study** (Agent-As-Tool vs. Plan-Execute) and a **hybrid scoring system**. We introduce an **automated failure discovery** procedure that identifies non-obvious reasoning and recovery bottlenecks.
- ♦ **Pillar III: Community & Open Source.** We operationalized AssetOpsBench as a **live competition** on Codabench[58], yielding 350+ users, 500+ submissions and 881+ trajectories. We release all containerized interfaces and starter templates to ensure the benchmark serves as a diagnostic utility for the broader research community.

Finally, we experimented with an additional **closed-source 162 scenarios** to demonstrate generality, spanning 10 asset classes, 53 failure modes, and 20 sensors. These include 42 live-production-deployment scenarios (>90% correctness verified by a domain expert), 17 hydraulic system, 15 metro train, and 88 failure-mode scenarios encompassing diverse asset–failure–sensor relationships.

2 Related Work

Generalist Agents. In this section, we review prior work on agent architectures and benchmarks relevant to ASSETOPS BENCH, focusing on generalist agents that orchestrate sub-agents to solve complex tasks, as seen in web-based systems like Magentic [10] and CUGA [29], multimodal frameworks like GEA [45], and software engineering platforms such as HyperAgent [18], ChatDev [37], and MetaGPT [13]. These systems typically rely on fixed sub-agents (e.g., browsers, editors) and hard-coded reasoning paradigms like ReAct or plan-execute. Consequently, they often lack the flexibility to adapt to novel tasks or support alternative coordination strategies like AOP [26] and Prospector [25].

Domain and Application-Specific Agents. Specialized benchmarks like MLE-bench [4], MLAGentBench [16], and MLGym [32] evaluate agents on scientific discovery and machine learning workflows. Similarly, application-focused frameworks like ITBench [23] and AIOpsLab [6] replicate site reliability engineering and system auditing. However, these benchmarks often prioritize tabular or image data, lacking the integrated support for the hybrid temporal and textual modalities critical for physical asset monitoring. Furthermore, they frequently lack the composability needed to evaluate agents across diverse, cross-modal industrial environments.

Action-Oriented Models. Performance is increasingly enhanced via fine-tuned models or *Large Action Models (LAMs)* designed for grounded execution, including TaskBench [41], xLAM [62], Agent-Gen [14], AgentBank [44], AgentRM [55], FireAct [5], and Action-Studio [61]. While effective in Windows-based [48], programming, or web environments, these models have yet to demonstrate applicability to the complex reasoning required for industrial automation with hybrid agent compositions.

Open Challenges. Despite recent advances, several critical gaps remain in the industrial agentic AI landscape. We categorize these into three primary deficiencies:

- (1) **Domain-Specific Data Scarcity:** There is a profound lack of comprehensive benchmarks targeting industrial asset domains, particularly for tasks involving condition-based monitoring and work order planning. To quantify this gap, we conducted a systematic analysis of a **catalog of 135 public industrial datasets** [8]. Our findings reveal that:
 - Only **1/135** datasets include any work-order or operational context.
 - Only **39%** (53/135) mention failure modes, typically limited to 1 or 2 modes.
 - **0%** of these datasets support agentic applications or integrated sensor-history-to-action workflows.
- (2) **Underrepresentation of Time-Series Modalities:** While time-series data is the backbone of infrastructure monitoring, it remains underrepresented in existing agentic benchmarks, which favor text-heavy or web-navigation tasks.
- (3) **Orchestration Complexity:** Few systems support orchestration across heterogeneous agents, such as blending text, code, and simulation, nor do they offer modular reasoning strategies adaptable to high-stakes, multi-agent workflows.

Addressing these gaps is essential for advancing general-purpose agent intelligence in real-world industrial domains.

3 Intelligent Agent-Based Asset Operations

Industrial asset operations involve complex, heterogeneous workflows where experts must interpret multi-modal sensor data and make timely decisions. Interdependent tasks, such as *root cause analysis*, *predictive maintenance*, and *work order bundling*, require reasoning across historical telemetry, asset metadata, and operational constraints. This necessitates intelligent agents capable of decomposing high-level intent into structured, executable recommendations. A representative complex workflow is illustrated by the following user request:

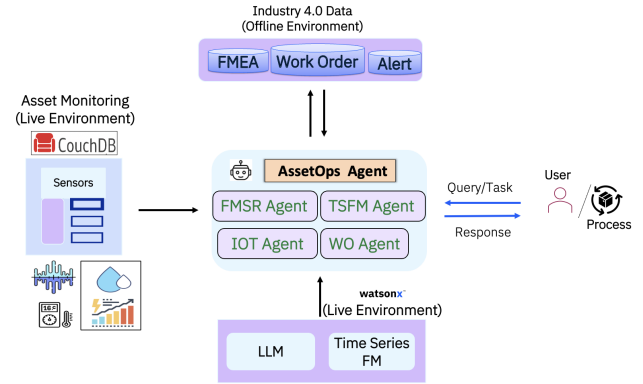


Figure 2: ASSETOPS BENCH Simulated Environment.

“Help **configure** an **anomaly detection** model to monitor power consumption of CUXP and trigger **alerts** when usage is **projected to exceed 8 Watts** above the maximum deviation observed over the past 30 days.”

Such scenarios facilitate critical corrective actions, such as automated service request creation. The diversity of these tasks, spanning data interpretation, anomaly reasoning, and operational decision-making, underscores the need for a coordinated multi-agent framework designed for the rigors of industrial workflows.

To address these demands, we operationalize a multi-agent framework within a **simulated Docker environment** (Figure 2), centered on a global coordinator: the **AssetOps Agent**. While general-purpose systems [10] often rely on generic sub-agents like web-surfers or terminal handlers, our architecture utilizes specialized, domain-inspired modules: *IoT* for telemetry, *FMSR* for failure mode mapping, *TSFM* for foundation model-driven time-series analysis, and *WO* for work-order modeling. This containerized sandbox provides agents with secure, standardized API access to Industry 4.0 data, including FMEA logs, alerts, and telemetry, facilitating reproducible end-to-end execution. Furthermore, the environment natively supports diverse orchestration paradigms, such as *Agent-As-Tool* and *Plan-Execute*, enabling comparative research into how distinct reasoning strategies handle complex, real-world queries.

To ensure ASSETOPS BENCH reflects operational reality, we move beyond the **API-driven** scenario generation typical of prior work [41, 59] in favor of a domain-authentic, **intent-driven** approach. Grounded in ISO 55000 [22] and ISO 14224 [21] standards, we established a four-stage taxonomy to guide the scenario generation process, mirroring the lifecycle of physical asset management:

Phase I: Asset Configuration. Focuses on foundational activities such as retrieving Failure Mode and Effects Analysis (FMEA) documentation and defining performance KPIs.

Phase II: Model Selection and Analysis. Agents apply anomaly detection models and leverage LLM-powered retrieval to surface critical historical failures from multi-modal records.

Phase III: Monitoring and Execution. Mimics live operations, requiring agents to manage real-time telemetry, refine detection parameters, and enforce safety guardrails.

Phase IV: Maintenance and Response. Focuses on actionable outputs, including automated work-order generation, system health summarization, and intervention.

Table 1: Key Data Modalities: Overview of the multi-source foundation used for scenario construction

Data Source	Example Fields
Sensor Telemetry Assets: 6 Quantity: 2.3M pts	Chiller Return Temp. Chiller % Loaded Condenser Water Flow
FMEA Records Assets: 3 Quantity: 53 entries	Failure Location / Component Degradation Mechanism Degradation Influences
Work Orders Assets: 10+ Quantity: 4.2K records	ISO Failure Code Event Log Timestamp Linked Anomaly / Alert

By grounding scenario definitions in international ISO standards, we achieve a level of platform-agnosticism that allows findings to generalize across diverse industrial ecosystems, from Oracle to IBM Maximo [19, 35]. This shift from “synthesizing tasks around tools” to “aligning tools with industrial intent” represents a significant step toward evaluating agents for true operational reliability.

4 ASSETOPS BENCH: Dataset, Scenarios, and Agents

ASSETOPS BENCH is a comprehensive benchmarking ecosystem featuring 141 expert-curated scenarios and specialized AI agents integrated with a multi-source industrial dataset.

4.1 Multi-Source Dataset

As detailed in Table 1, the benchmark incorporates over **2.3 million sensor data points** across six primary assets, comprising four chillers and two Air Handling Units (AHUs). These streams are ingested from Building Management Systems (BMS) [52] and SkySpark [43], capturing critical time-series signals such as return temperatures and condenser flow rates. To provide a diagnostic “ground truth,” we include structured failure models derived from **Failure Mode and Effects Analysis (FMEA)** records sourced from the Reliability Strategy Library [20]. These 53 entries provide deep insights into the physical mechanisms of failure, such as *wear* or *erosion*, and the influencing stressors, such as *shock loading*, that an agent must interpret to provide accurate recommendations.

Furthermore, ASSETOPS BENCH integrates a longitudinal history of **4.2K work orders** spanning 11 years, extracted from IBM Maximo systems. These records are enriched with ISO-standard failure codes and event timestamps, allowing agents to correlate historical maintenance actions with detected anomalies. Collectively, this foundation comprises nine distinct modalities, including sensors, work orders, alerts, FMEA, and expert-derived rules, to facilitate a comprehensive evaluation of multi-hop reasoning in high-stakes industrial environments.

4.1.1 Data Quality Analysis. Table 2 reports the per-sensor data-quality analysis over the benchmark dataset (2 AHUs, 4 Chillers), spanning normal operation, expected high-zero behavior, transient outliers, sensor anomalies, and communication gaps.

Table 2: Per-sensor data-quality analysis on the ASSETOPS BENCH dataset (2 AHUs, 4 Chillers). Outliers are flagged using $|z| > 3.0$ and a $1.5 \times \text{IQR}$ rule.

Data phenomenon	Sensors	Observed rate
Normal operation	Temperature sensors	40–50% actual data
Expected high-zero	Schedule-driven control valves	80–97% zeros
Transient outliers	Startup / shutdown sensors	3–10% outliers
Sensor anomalies	Liquid refrigerant temperature	5–18% outliers
Communication gaps	Run status, control signals	30–97% missing

4.2 Scenario Design and Coverage

Each scenario in ASSETOPS BENCH represents a structured operational query grounded in our ISO-aligned task taxonomy and multi-source datasets. We formalize each scenario as a 5-tuple:

$$P = \langle id, type, text, category, characteristic form \rangle$$

where *id* is a unique identifier and *category* specifies the functional requirement, such as knowledge retrieval or analytical reasoning. The *text* field contains the natural language query, which is specifically designed to reflect the **intent patterns** of industrial operators. Rather than referencing database schemas, these queries utilize physical operational terms, such as *energy consumption* or *chiller performance*, requiring the agent to map human intent to the underlying data modalities. The *type* denotes the operational domain, including specialized modules (IoT, FMSR, TSFM, WO) or end-to-end tasks, while *characteristic form* defines the expected output, such as an executable API call or a multi-step action plan.

Scenario # 507

Text : What is the predicted energy consumption for Chiller 9 in the week of 2020-04-27 based on data from the MAIN site?

Type : End-to-End: TSFM Agent, IoT Agent

Category : Analytical

Characteristic Form : The expected response should confirm the successful execution of all required actions, ensuring that the correct asset **Chiller 9**, location **MAIN**, and time range week of **2020-04-27** were used for data retrieval and analysis. It should specify that the agent identified the sensor name **power input sensor** and retrieved the historical energy consumption data for Chiller 9 during the specified time period. The response must also explain that the agent attempted to analyze the data for energy consumption prediction, but was unable to do so due to **insufficient data**, as the power input for Chiller 9 was consistently **0.0** from **2020-04-20** to **2020-04-25**, indicating that the chiller was **not operating**.

Figure 3: An example of an expert-authored query.

Figure 3 illustrates **Utterance 507**, a representative case requiring the agent to forecast future energy consumption. To resolve this query, the agent must map high-level user intent to specific

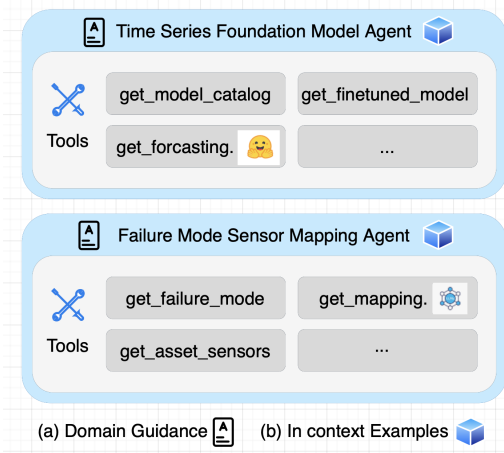


Figure 4: Representative tools for time-series and failure-mode reasoning agent

operational variables, such as power input, while navigating industrial data artifacts like high zero-value density. Crucially, the agent is expected to recognize that the chiller was non-operational during the specified window; it must then correctly conclude that data is insufficient for forecasting rather than generating a spurious result. This scenario demonstrates how our expert-designed tasks evaluate an agent’s ability to navigate the ambiguity of **operator-centric language** and physical state logic rather than merely assessing tool-calling syntax.

Beyond individual tasks, the **11-year longitudinal history** of work orders provides the temporal depth required for high-level strategic synthesis. This scale enables the evaluation of long-term lifecycle strategies, such as determining if year-over-year increases in corrective maintenance justify capital replacement over continued repair. Scenarios **IDs 442–446**, available on our github repository, specifically leverage this breadth to assess strategic oversight through trend analysis and probability forecasting across the asset’s operational lifespan.

Development Effort. To ensure operational authenticity, 141 scenarios were synthesized through an **18-month collaboration** with reliability engineers to capture realistic fault signatures and cross-sensor interactions across assets like AHUs and chillers. As shown in Figure 1b, the benchmark comprises 99 **single-agent utterances** targeting specific domain modules and 42 **multi-agent tasks** necessitating coordinated reasoning and data exchange.

4.3 Agents and Implementation

ASSETOPSBENCH features four specialized domain agents: **IoT**, **TSFM**, **WO**, and **FMSR**. To navigate the complexity of industrial telemetry and metadata, these agents utilize a suite of over 15 distinct tools. As shown in Figure 4, the **TSFM** agent integrates a pretrained time-series foundation model from Hugging Face for forecasting [30], while the **FMSR** agent utilizes an LLM-driven `get_mapping` function to bridge the semantic gap between physical failure modes and sensor tags [7]. Methodologically, the TSFM, IoT, and FMSR agents employ the ReAct [38, 60] paradigm, whereas the WO agent utilizes CodeReAct [49, 63] to handle the structured logic required for maintenance record analysis.

To coordinate these heterogeneous modules, we implement a global **AssetOps Agent** that supports two primary orchestration paradigms:

- **Agent-As-Tool:** Domain agents are registered as high-level tools within a supervisor agent. This supervisor uses ReAct to iteratively select and query specific agents, emulating the layered decision-making of hierarchical industrial organizations.
- **Plan-Execute:** A specialized **Planner** and **Reviewer** decompose the user query into a Directed Acyclic Graph (DAG). An **Orchestrator** then executes the DAG, utilizing a shared memory module to transfer context between agents. This approach adapts the ReWoo [56] framework with an additional verification layer inspired by [26].

These components, comprising specialized tools, agents, and their orchestration, are implemented using open-source toolkits [29, 34, 46], and our agent selection and coordination strategies follow modular architectural trends established in recent agentic benchmarking literature [28, 50]. Crucially, these implementations are not naive compositions of existing frameworks: five systems-level innovations, such as a dual-output interface and the *Agent-Ask* recovery protocol (which raises task completion from 59% to 66%), are required for reliable long-horizon reasoning. We provide an in-depth discussion of these innovation of orchestration methodology in Appendix A.1.

5 Experiments and Leaderboard

To evaluate LLM capabilities for industrial automation within ASSETOPSBENCH, we benchmark seven state-of-the-art models across the two orchestration paradigms from Section 4.3. Following recent frameworks [1, 46, 48, 51], we adopt a dual-evaluation strategy: a **rubric-based** assessment of reasoning quality and a **reference-based** mechanism to verify ground-truth alignment [3, 51, 59].

5.1 Evaluation Methodology

Rubric-based LLM-As-Judge Scoring. Each scenario is paired with a *characteristic form* (C), which is a structured specification of expected outputs and procedural steps serving as the **soft ground truth**. We formalize the evaluation via a scoring function $\Phi(Q, T, C) \rightarrow \{y_1, y_2, y_3\}$, where Q is the query and T is the agent trajectory. The function yields three scalar scores $y_i \in [0, 1]$ for (1) **Task Completeness** (y_1), (2) **Data Retrieval Accuracy** (y_2), and (3) **Result Verification** (y_3). We employ llama-4-maverick as the primary judge; human validation of this judge is provided in Section 5.5.

Reference-Based Scoring. To evaluate planning and decomposition rigor, we establish a structured ground-truth reference for each scenario comprising: (1) `planning_steps` for strategic intent, (2) `execution_steps` for tool-calling actions, and (3) causal dependencies via `execution_links`. We quantify the alignment between these benchmarks and the agent-generated reasoning traces or Directed Acyclic Graphs (DAGs) using the **ROUGE** metric to assess lexical and structural overlap [41, 59].

5.2 Experimental Setting

To quantify agent effectiveness, we adopt the **Pass^k** metric [59]. Unlike Pass@k, which requires only one success in k attempts,

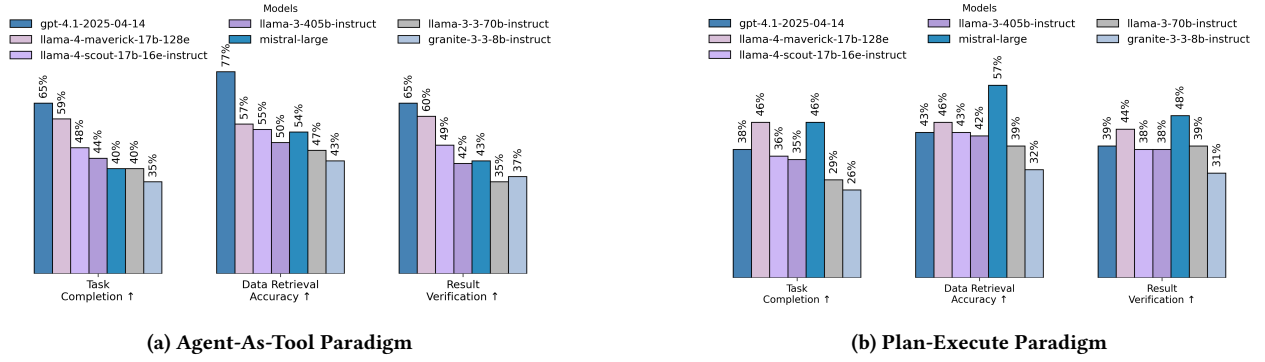


Figure 5: LLM-As-Judge scoring results ranked by task completion rate (y_1). See Table 3 for additional frontier models.

Pass^k estimates the probability of succeeding across *all* k trials. This stricter criterion aligns with industrial reliability standards where consistent behavior is mandatory and stochastic retries are operationally impractical [39, 59]. We report Pass^1 by default, representing the success rate over single task instances.

Parameters. To ensure stable performance estimates for our rubric-based evaluation, the judge agent is executed five times per trajectory, and we report the average. All specialized LLM agents operate with a sampling temperature of 0 to prioritize deterministic reasoning, while the evaluation judge utilizes a temperature of 0.3 to allow for nuanced qualitative assessment.

Models. We include closed-source models (e.g., gpt-4.1), frontier open-source models (e.g., llama-4-maverick, llama-4-scout, mistral-large, llama-3-405b), and medium-to-small open-source models (e.g., llama-3-70b, granite-3-8b). We release a public leaderboard for AssetOpsBench and will continuously update it as new models and budget become available to our ecosystem.

5.3 AssetOpsBench Leaderboard

Rubric-based Performance Analysis. Figure 5 summarizes performance across three dimensions: task completion, data retrieval accuracy, and result verification. ASSETOpsBENCH remains significantly challenging for current LLMs, with no model exceeding a 70% completion rate. Under the **Agent-As-Tool** setup (Figure 5a), gpt-4.1 leads with the **highest task completion (65%) and retrieval accuracy (77%)**. Among open-source candidates, llama-4-maverick is the most competitive, matching the leading models in reasoning stability.

In contrast, **Plan-Execute** paradigm (Figure 5b) exhibits a significant performance regression. The peak completion rate drops to 46%, suggesting that the “plan-ahead” pattern introduces cascading failure modes in industrial contexts. Most strikingly, gpt-4.1 shows the sharpest decline (**65%→38% completion**), indicating that its proficiency in iterative tool use does not generalize to rigid planning interfaces. Conversely, mistral-large and llama-4-maverick emerge as the most robust for Plan-Execute workflows: mistral-large achieves the highest retrieval (57%) and verification (48%) scores, while llama-4-maverick ties for the top completion rate. Due to its balanced performance across both paradigms, we adopt llama-4-maverick as the default LLM for our ablation studies.

5.4 Extended Frontier-Model Leaderboard

The original submission evaluated several frontier models, including Claude, Gemini, and GPT variants; model selection is primarily driven by the availability of frontier models within our monthly evaluation budget. To broaden this comparison, we extended our experiments with four additional frontier models under both orchestration paradigms. Table 3 reports the results.

Table 3: Extended frontier-model leaderboard under both orchestration paradigms. TC: task completion; DRA: data retrieval accuracy; RV: result verification. Best result per column within each paradigm is in bold.

Model	TC (%)	DRA (%)	RV (%)
<i>Agent-As-Tool</i>			
gpt-oss-120b	46.10	46.10	46.10
gpt-5	52.48	55.32	58.16
gemini-2.5-pro	42.55	53.19	43.97
claude-4-sonnet	63.83	75.89	66.67
<i>Plan-Execute</i>			
gpt-oss-120b	27.66	40.43	27.66
gpt-5	36.88	41.84	36.17
gemini-2.5-pro	33.33	60.99	32.62
claude-4-sonnet	64.54	79.43	63.12

Several observations stand out. First, claude-4-sonnet competes with the previously reported leaders (llama-4-maverick, gpt-4.1) and the other frontier models in this batch, establishing a new state of the art on ASSETOpsBENCH. Second, it is the only model whose performance does not degrade under Plan-Execute (task completion 63.83% → 64.54%); every other frontier model we tested suffers a substantial regression, consistent with our earlier observation that the plan-ahead pattern introduces cascading failure modes. Third, higher planning accuracy does not translate into higher execution accuracy: gemini-2.5-pro and gpt-5 rank near the top on planning quality but trail claude-4-sonnet by more than 20 points on task completion.

Table 4: Execution efficiency summary: mean steps per tasks and mean runtime per tasks

Model	Agent-As-Tool				Plan-Execute			
	Single-agent tasks		Multi-agent tasks		Single-agent tasks		Multi-agent tasks	
	Steps	Runtime (Sec)	Steps	Runtime (Sec)	Steps	Runtime (Sec)	Steps	Runtime (Sec)
gpt-4.1	6.0	104	6.4	218	2.6	93	2.9	180
mistral-large	4.9	347	5.2	289	2.7	187	3.0	210
llama-3-405b-instruct	4.8	250	5.6	255	3.1	208	4.0	224
llama-3-70b-instruct	3.9	101	4.3	151	6.7	382	6.5	370
llama-4-maverick-17b-128e	4.3	120	4.5	137	4.0	385	3.9	377
llama-4-scout-17b-16e	4.4	101	5.8	178	3.9	172	4.4	218
granite-3-3-8b	5.3	197	6.6	228	5.2	413	5.1	433

5.5 Operational Efficiency and Evaluation Reliability

Execution Efficiency. Table 4 reports the mean number of steps (tool invocations) and wall-clock runtime per task for seven LLMs under two agentic implementations. Overall, **Plan-Execute** is consistently more step-efficient: most models complete tasks in ≈ 2.6 – 4.4 steps, whereas **Agent-As-Tool** typically requires ≈ 4 – 6 steps due to its iterative *think-act-observe* loop. However, Plan-Execute exhibits limited sensitivity to task complexity: the step counts for single-agent and multi-agent tasks are often nearly identical (e.g., gpt-4.1: 2.6 vs. 2.9; granite-3-3-8b: 5.2 vs. 5.1), suggesting a tendency to *over-plan* even when the objective is relatively simple.

Runtime trends are more nuanced than step counts (See Runtime columns in Tables 4). While fewer steps can reduce execution time for some models (e.g., gpt-4.1 remains the fastest and most step-efficient Plan-Execute backbone, especially on multi-agent tasks), several models incur substantial overhead in Plan-Execute (e.g., llama-3-70b-instruct and granite-3-3-8b exceed ~ 370 – 430 seconds on multi-agent tasks). In contrast, Agent-As-Tool can be competitive—and for some backbones, dramatically faster—despite taking more steps (e.g., llama-4-maverick drops from ~ 377 s in Plan-Execute to ~ 137 s in Agent-As-Tool on multi-agent tasks), indicating lower per-step overhead and more direct tool control. At the same time, tool/data effects can dominate end-to-end latency: mistral-large shows high variance in Agent-As-Tool due to a pathological outlier with prolonged JSONReader calls over large histories.

Overall, Plan-Execute offers structural and step efficiency, but its wall-clock cost depends strongly on model-planner-executor alignment, whereas Agent-As-Tool often spends more steps yet tends to deliver stronger end-task quality when correctness is prioritized.

Small Language Models (SLM) Analysis. Overall, SLMs tend to underperform. However, we conduct a more detailed analysis to identify where SLMs are effective and where they break down. In the Agent-As-Tool evaluation, models such as granite-3-8b and llama-3-3-70b exhibit weaker aggregate performance, yet they demonstrate clear areas of specialization, as shown in Figure 6a. Both models perform strongly on structured sensing and diagnostic tasks: for example, granite-3-3-8b achieves 15/20 on IoT, 18/22 on FMSR, and 19/23 on TSFM, while llama-3-3-70b reaches 12/20, 18/22, and 20/23 on the same categories. However, they struggle

substantially on Work Order tasks, with scores of only 2/36 and 7/36, indicating that procedural, multi-step coordination remains difficult even under the Agent-As-Tool mechanism. This underscores a key insight: industrial deployments may benefit most from **hybrid LLM-SLM agent architectures**, where strong specialists handle sensing and diagnostics while more capable generalist models manage planning and end-to-end coordination [2].

Human Validation of LLM Judge. To assess the reliability of using LLMs as automatic evaluators for benchmarking tasks, we compare model-generated judgments against human annotations on a sample of 40 tasks. Each task is evaluated along three dimensions by four domain experts, all operating under the same information constraints as the LLMs. Before selecting a default evaluator, we compared several candidate judge models; gpt-4.1 showed only moderate alignment with expert assessments (69% accuracy, Cohen’s $\kappa = 0.44$). In contrast, llama-4-maverick provided substantially stronger agreement with human judgments and was therefore selected as the default judge model for the main analysis. Inter-rater reliability indicates substantial agreement across dimensions, with *Data Retrieval Accuracy* exhibiting the strongest consistency (Cohen’s $\kappa = 0.79$, 90.48% accuracy). *Task Completion* ($\kappa = 0.62$) and *Generalized Result Verification* ($\kappa = 0.71$) also show high alignment among evaluators.

Cross-model validation of LLM Judge. To assess the robustness of our LLM-as-Judge protocol, we ran a preliminary cross-model agreement study between our primary judge, Llama-4 Maverick, and claude-4-sonnet as an alternative strong judge, across all evaluated industrial scenarios. Full human re-validation and the corresponding κ agreement against expert labels for the new judge are left for future work, but we can already report judge-judge agreement. The two judges reach the same binary (Pass/Fail) verdict in more than four of every five scenarios, with a raw accuracy of 81.29% and a Cohen’s κ of 0.6267. This κ falls within the *substantial* agreement range (0.61–0.80), indicating that the consensus between the two independent judges is well beyond chance. Together with the human-validation results already reported in the paper, this supports the robustness and reproducibility of our evaluation protocol. Since claude-4-sonnet also performs strongly across both orchestration paradigms.

Further analyses, including ablations on in-context examples and a single-agent baseline, alongside cost and evaluation-stability studies, are provided in Appendix A.2–A.5.

5.6 Scenarios Realism Analysis

To validate the operational authenticity of ASSETOPS BENCH, we conducted a formal expert validation study. A panel of domain experts—including reliability engineers, maintenance specialists, and data scientists—evaluated a random sample of 25 representative scenarios. The results demonstrate high statistical robustness. The study yielded a **Cronbach’s Alpha of 0.89**, confirming strong internal consistency across the scenario ratings. Although we observed natural variability of individual raters ($ICC(1) = 0.23$), aggregate reliability was high ($ICC(2) = 0.88$), indicating that averaged expert judgments provide a highly stable estimate of perceived realism. Furthermore, a Fleiss’ Kappa of 0.21 was obtained; while seemingly modest, this level of agreement is consistent with subjective realism assessments in complex, high-stakes industrial domains. These findings verify that our 141 scenarios faithfully encapsulate the diagnostic reasoning and technical rigor required in professional industrial practice.

5.7 Failure Analysis via Agent Trajectories

We analyze agent reasoning traces (trajectories) to identify systemic failure modes. Across approximately 881 collected trajectories for the Agent-As-Tool strategy, we conduct a two-pronged error analysis focusing on **tool-related errors** and **agent failure modes**.

Failure Analysis on Tool Use. Each agent step in a trajectory is logged as a structured JSON record capturing the *action type* and *execution state*. At the sub-agent level, we distinguish between **Tool-oriented actions**, which invoke predefined functions with well-defined inputs and outputs, and **CodeReAct-oriented actions**, where agents dynamically generate and execute Python code. Our analysis shows that Tool-oriented actions achieve higher valid-execution rates, whereas CodeReAct-oriented actions incur more runtime failures due to the variability of the generated code. Tool-oriented failures are concentrated in a small number of tools, including `jsonreader`, `tsfm_integrated_tsd`, and `Read Sensors From File`, highlighting challenges related to input validation and hallucinated parameter passing.

Emerging Failure Modes Discovery. Now we investigate trajectories from a semantic perspective. Recent work [3] defines 14 failure modes for agent trajectories. Figure 6b shows the distribution of the failure mode in our 881 trajectories in this taxonomy. We found that **system design** is the most common source of failures. This taxonomy provides guidance to improve agent development. For instance, since the “Fail to Ask for Clarification” mode occurs around 10% of the time, we introduced a feature in the Agent-As-Tool strategy that allows sub-agents to ask the parent agent a clarification questions at any point during execution. We reran the entire benchmark on the default LLM, and this change led to significant performance improvements for `llama-4-maverick`, increasing task completion from 59% to 66%, surpassing `gpt-4.1`. To capture failure mode behaviors beyond this taxonomy, we allowed self-discovery of up to two **novel failure modes** per trace, revealing *emergent and compound failures* not covered by existing classifications. This automated diagnostic find several emergent failure modes: **Overstatement of Task Completion** (122 cases, 23.8%), **Extraneous or Ambiguous Output Formatting** (110 cases, 21.4%) and **Ineffective Error Recovery** (160 cases).

5.8 Causal Analysis of Execution Traces.

To move from *what* fails to *why*, we complement the taxonomy above with a causal study of execution traces. For each trajectory we extract structured features describing how the agent executed the task: tool usage, trajectory length, repetition, loop behavior, execution cost, and error signals. We combine three analyses to identify which features predict success, namely descriptive statistics, regression, and group-based comparison, and apply DirectLiNGAM [42] to recover the causal ordering among them. Applied to 56 trajectories, the analysis reveals that although the nominal success rate is high (94.6%), 73% of runs are inefficient and 76.7% contain errors, meaning most runs succeed despite suboptimal behavior. Regression shows that longer executions (tokens, wall-clock time) and errors reduce the probability of success, while more structured reasoning (average thought length) increases it, with a coefficient of +0.88. DirectLiNGAM recovers a consistent ordering: early execution instability (repetition and loop signals) drives trajectory expansion (more steps and tool calls), which increases execution cost (time and tokens), which in turn raises error rates and reduces reliability. Three findings emerge: repetition is the key upstream driver of the cascade; errors are downstream symptoms rather than root causes; and the primary failure mechanism is inefficiency (redundant actions and looping) rather than a lack of reasoning capability. The practical implication is that improving agent control flow, by suppressing loops and redundant tool calls, would yield larger gains than further investment in reasoning quality alone.

6 Community Adoption and Generalization

Community Engagement. To move beyond static evaluation [17], we operationalized ASSETOPS BENCH as a live, code-executed competition [36] on **Codabench**. This platform enables end users to integrate custom agent architectures and evaluate them against hidden scenarios via containerized submissions. Over a three-month period, the competition engaged 350+ participants (44.2% Undergraduate, 28.4% Industry, 23.6% Postgraduate, and 3.8% Other), resulting in 500+ agent submissions across specialized planning and execution tracks. This deployment demonstrates the system’s capability to host custom agents and maintain a dynamic leaderboard; furthermore, to maximize academic impact and inclusivity, we provided full LLM API support funded through our internal budget to ensure participants could iterate without financial barriers.

Generalization Across Industrial Assets. To evaluate generalization capabilities, we developed 162 additional scenarios across four distinct datasets and an extended environment (Figure 2) with new datasets and scenarios. These datasets include MetroPT-3 [9, 47] (15 scenarios) focused on compressor faults, the UCI Hydraulic System [12] (17 scenarios) for hydraulic component failures, an internal Asset Health dataset (42 scenarios) derived from industrial work orders, and FailureSensorQA (88 scenarios) based on ISO-standardized sensor-to-failure mapping. Figure 7 shows summary of results for Agent-As-Tool approach. Among all the datasets, scenarios of MetroPT-3 are difficult as we observed poor performance (task completion rate = 26.7%), whereas Asset Health (a production usecase) had a near perfect solution.

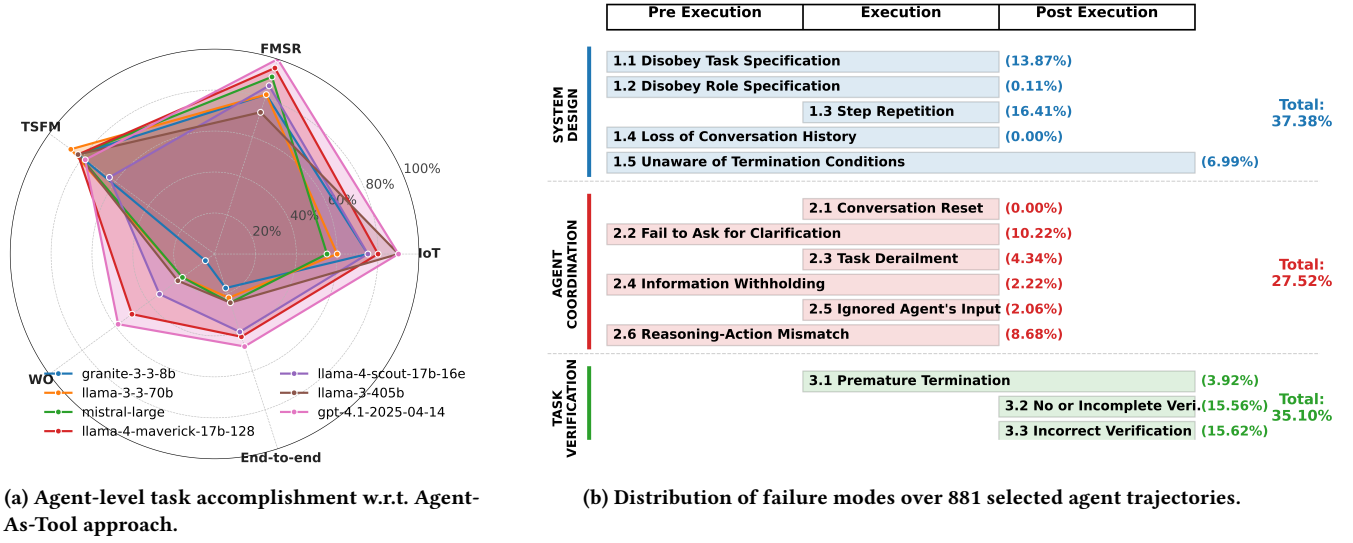


Figure 6: Agent performance and failure-mode coverage in AssetOpsBench.

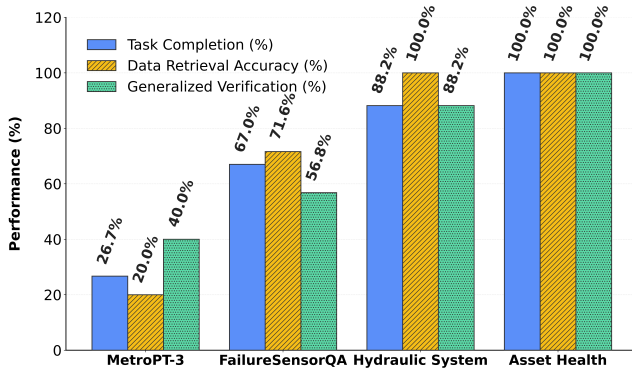


Figure 7: Extended Validation for Different Asset Class

6.1 Optimization-Aware Generalization

Beyond the framework generality shown in Figure 7, where new asset classes, datasets, and tasks are introduced while keeping all tools, prompts, and in-context examples unchanged, we ask whether ASSETOpsBENCH can support *learning* from prior experience. Using the Agent-As-Tool paradigm with llama-4-maverick and DSPy’s BootstrapFewShot optimizer [24], we study generalization along two axes: a temporal axis (learning from past incidents) and a cross-asset axis (learning from related assets). In both cases the optimizer learns only from a small set of successful demonstrations; no tools, prompts, or agent code are modified.

Temporal generalization (MetroPT-3). We use a time-based split over 17 scenarios: the 5 earliest are used for optimization and the 12 later scenarios are held out for validation (Table 5, top). Learning from past trajectories yields large gains on the two dimensions that most reflect end-to-end reasoning: task completion and result verification each rise by 22.2 points (67% relative).

Cross-asset transfer (FailureSensorIQ). We optimize on 15 scenarios drawn from two asset classes and validate on the remaining 73, under the same setup (Table 5, bottom). Here the optimizer improves all three rubric dimensions simultaneously (+4.2, +4.7, +5.2 points), indicating that demonstrations collected on two asset classes transfer cleanly to unseen ones.

Table 5: Optimization-aware generalization along a temporal axis (MetroPT-3) and a cross-asset axis (FailureSensorIQ). TC: task completion; DRA: data retrieval accuracy; RV: result verification. Best result per dataset block is in bold.

Dataset	Setting	TC	DRA	RV	Avg.
MetroPT-3	Baseline	33.33	91.67	33.33	60.00
	Optimized (DSPy)	55.56	86.11	55.56	65.00
FailureSensorIQ	Baseline	74.18	73.71	62.44	79.72
	Optimized (DSPy)	78.40	78.40	67.61	83.94

7 Conclusion and Future Outlook

This paper introduces ASSETOpsBENCH, a formalized framework for industrial AI agents using a standardized evaluation methodology and the Agent-As-Tool paradigm to orchestrate complex multi-agent interactions. Through a competition that engaged 350+ participants across academia and industry, we demonstrated the system’s capability to host custom agents and foster iterative refinement via reproducible, code-executed evaluations. Future extensions will incorporate realistic constraints, such as compute limits and API costs, to drive the development of cost-aware autonomous industrial algorithms. We plan to generalize framework to support the additional tasks such as work order classification, problem code classification, etc.

References

- [1] Pierre Andrews, Amine Benhalloum, Gerard Moreno-Torres Bertran, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, Virginie Do, Romain Froger, Emilien Garreau, Jean-Baptiste Gaya, Hugo Laurençon, Maxime Lecanu, Kunal Malkan, Dheeraj Mekala, Pierre Ménard, Grégoire Mialon, Ulyana Piterberg, Mikhail Plekhanov, Mathieu Rita, Andrey Rusakov, Thomas Scialom, Vladislav Vorotilov, Mengjue Wang, and Ian Yu. 2025. ARE: Scaling Up Agent Environments and Evaluations. arXiv:2509.17158 [cs.AI] <https://arxiv.org/abs/2509.17158>
- [2] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. 2025. Small Language Models are the Future of Agentic AI. arXiv:2506.02153 [cs.AI] <https://arxiv.org/abs/2506.02153>
- [3] Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail?. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. <https://openreview.net/forum?id=fAjyBmnmr>
- [4] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Aleksander Madry, and Lilian Weng. 2025. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. In *The Thirtieth International Conference on Learning Representations*. <https://openreview.net/forum?id=6s5uXNWGIlh>
- [5] Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik R Narasimhan, and Shunyu Yao. 2024. FireAct: Toward Language Agent Finetuning. <https://openreview.net/forum?id=RqUMWdDg52>
- [6] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. 2025. AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds. *CoRR* abs/2501.06706 (2025). arXiv:2501.06706 doi:10.48550/ARXIV.2501.06706
- [7] Christodoulos Constantinides, Dhaval C Patel, Shuxin Lin, Claudio Guerrero, Sunil Dagajirao Patil, and Jayant Kalagnanam. 2025. FailureSensorIQ: A Multi-Choice QA Dataset for Understanding Sensor Relationships and Failure Modes. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. <https://openreview.net/forum?id=9KfKMAy2ut>
- [8] Jonathan Dambros. 2025. awesome-industrial-datasets: A curated collection of public industrial datasets. GitHub repository. <https://github.com/jonathanwvd/awesome-industrial-datasets> Accessed: 2026-01-01.
- [9] Narjes Davari, Bruno Veloso, Rita P Ribeiro, Pedro Mota Pereira, and João Gama. 2021. Predictive maintenance based on anomaly detection using deep learning for air production unit in the railway industry. In *2021 IEEE 8th International Conference on Data Science and Advanced Analytics (DSAA)*. IEEE, 1–10.
- [10] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. 2024. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. *CoRR* abs/2411.04468 (2024). arXiv:2411.04468 doi:10.48550/ARXIV.2411.04468
- [11] Will Fu-Hinthorn. 2025. Benchmarking Multi-Agent Architectures. LangChain Blog. <https://blog.langchain.com/benchmarking-multi-agent-architectures/> Accessed: 2026-01-01.
- [12] Nikolai Helwig, Eliseo Pignatelli, and Andreas Schtze. 2015. Condition monitoring of hydraulic systems. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5CW21>
- [13] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VtmBAGCN7o>
- [14] Mengkang Hu, Pu Zhao, Can Xu, Qingfeng Sun, Jian-Guang Lou, Qingwei Lin, Ping Luo, and Saravan Rajmohan. 2025. AgentGen: Enhancing Planning Abilities for Large Language Model based Agent via Environment and Task Generation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1* (Toronto ON, Canada) (KDD '25). Association for Computing Machinery, New York, NY, USA, 496–507. doi:10.1145/3690624.3709321
- [15] Kung-Hsiang Huang, Akshara Prabhakar, Onkar Thorat, Divyansh Agarwal, Prafulla Kumar Choubey, Yixin Mao, Silvio Savarese, Caiming Xiong, and Chien-Sheng Wu. 2026. CRMArena-Pro: Holistic Assessment of LLM Agents Across Diverse Business Scenarios and Interactions. *Transactions on Machine Learning Research* (2026). <https://openreview.net/forum?id=EP1pe3Fx1x>
- [16] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. 2024. Benchmarking Large Language Models as AI Research Agents. <https://openreview.net/forum?id=N9wD4RFWY0>
- [17] Hugging Face. 2026. Community Evals: Because We're Done Trusting Black-Box Leaderboards Over the Community. <https://huggingface.co/blog/community-evals>. Blog post.
- [18] Phan Nhat Huy, Tien N Nguyen, and Nghi D. Q. Bui. 2025. HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale. <https://openreview.net/forum?id=PZF4RsPMBG>
- [19] IBM. 2026. IBM Maximo Application Suite. <https://www.ibm.com/products/maximo> Accessed: 2026-05-13.
- [20] IBM. 2026. Reliability centered maintenance software with IBM Maximo Application Suite. <https://www.ibm.com/products/maximo/reliability-centered-maintenance> Accessed: 2026-02-08.
- [21] International Organization for Standardization. 2016. ISO 14224:2016 Petroleum, petrochemical and natural gas industries — Collection and exchange of reliability and maintenance data for equipment. <https://www.iso.org/standard/64076.html>. Provides asset taxonomy and data structuring guidance. Accessed: 2026-05-30.
- [22] International Organization for Standardization. 2024. ISO 55000:2024 Asset management — Vocabulary, overview and principles. <https://www.iso.org/standard/83053.html>. Defines asset management lifecycle and taxonomy. Accessed: 2026-05-30.
- [23] Saurabh Jha, Rohan R. Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya Bhavya, Mudit Verma, Harshit Kumar, Hirokuni Kitahara, Noah Zheutlin, Saki Takano, Divya Pathak, Felix George, Xinbo Wu, Bekir O Turkkan, Gerard Vanloo, Michael Nidd, Ting Dai, Oishik Chatterjee, Pranjal Gupta, Suranjana Samanta, Pooja Aggarwal, Rong Lee, Jae wook Ahn, Debanjana Kar, Amit Paradkar, Yu Deng, Pratibha Moogi, Prateeti Mohapatra, Naoki Abe, Chandrasekhar Narayanaswami, Tianyin Xu, Lav R. Varshney, Ruchi Mahindru, Anca Sailer, Laura Shwartz, Daby Sow, Nicholas C. M. Fuller, and Ruchir Puri. 2025. ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. In *Forty-second International Conference on Machine Learning*. <https://openreview.net/forum?id=jP59zr1bZk>
- [24] Omar Khattab, Arnab Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A. Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=sY5N0zY5Od>
- [25] Byoungjip Kim, Youngsoo Jang, Lajanugen Logeswaran, Geon-Hyeong Kim, Yu Jin Kim, Honglak Lee, and Moontae Lee. 2024. Prospector: Improving LLM Agents with Self-Asking and Trajectory Ranking. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizi, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 14958–14976. doi:10.18653/v1/2024.findings-emnlp.879
- [26] Ao Li, Yuexiang Xie, Songze Li, Fugee Tsung, Bolin Ding, and Yaliang Li. 2025. Agent-Oriented Planning in Multi-Agent Systems. In *The Thirtieth International Conference on Learning Representations*. <https://openreview.net/forum?id=EqL4U6gyU>
- [27] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-Play Compositional Reasoning with Large Language Models. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=HtqnVSC3jq>
- [28] Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. 2025. MCP-Universe: Benchmarking Large Language Models with Real-World Model Context Protocol Servers. arXiv:2508.14704 [cs.AI] <https://arxiv.org/abs/2508.14704>
- [29] Sami Marreed, Alon Oved, Avi Yaeli, Segev Shlomov, Ido Levy, Offer Akrabi, Aviad Sela, Asaf Adi, and Nir Mashkif. 2025. Towards Enterprise-Ready Computer Using Generalist Agent. arXiv:2503.01861 [cs.DC] <https://arxiv.org/abs/2503.01861>
- [30] Natalia Martinez, Fearghal O'Donncha, Wesley M. Gifford, Nianjun Zhou, Dhaval C Patel, and Roman Vaculin. 2026. Adaptive Conformal Anomaly Detection with Time Series Foundation Models for Signal Monitoring.. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=7uFbs68MSI>
- [31] Claire Bizon Monroc, Ana Busic, Donatien Dubuc, and Jiamin Zhu. 2024. WFCRL: A Multi-Agent Reinforcement Learning Benchmark for Wind Farm Control. In *The Thirty-eighth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. <https://openreview.net/forum?id=ZRM4hpZ3ED>
- [32] Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Mikhail Plekhanov, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, Dieuwke Hupkes, Ricardo Silveira Cabral, Tatiana Shavrina, Jakob Nicolaus Foerster, Yoram Bachrach, William Yang Wang, and Roberta Raileanu. 2025. MLLGym: A New Framework and Benchmark for Advancing AI Research Agents. In *Second Conference on Language Modeling*. <https://openreview.net/forum?id=ryTr83DxRq>
- [33] Avisek Naug, Antonio Guillen, Ricardo Luna, Vineet Gundecha, Cullen Bash, Sahand Ghorbanpour, Sajad Mousavi, Ashwin Ramesh Babu, Dejan Markovikj, Lekhapriya D Kashyap, Desik Rengarajan, and Soumyendu Sarkar. 2024. SustainDC: Benchmarking for Sustainable Data Center Control. In *Advances in Neural Information Processing Systems*, A. Globerson,

- L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 100630–100669. https://proceedings.neurips.cc/paper_files/paper/2024/file/b6676756f8a935e208f394a1ba47f0bc-Paper-Datasets_and_Benchmarks_Track.pdf
- [34] NVIDIA. 2025. About NVIDIA Agent Intelligence Toolkit Workflows, Version 1.1.0. <https://docs.nvidia.com/aiqtoolkit/1.1.0/workflows/about/index.html>. Accessed: 2025-11-23.
- [35] Oracle. 2025. IoT Platform. <https://www.oracle.com/cloud/cloud-native/iot-platform/>. Oracle IoT Asset Monitoring Cloud Service, Accessed: 2026-01-01.
- [36] Dhaval Patel, Nianjun Zhou, Shuxin Lin, James Rayfield, Chathurangi Shyalika, and Suryanarayana Reddy Yarrabothula. 2026. AssetOpsBench-Live: Privacy-Aware Online Evaluation of Multi-Agent Performance in Industrial Operations. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 48 (Mar. 2026), 41658–41660. doi:10.1609/aaai.v40i48.42372
- [37] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 15174–15186. doi:10.18653/v1/2024.acl-long.810
- [38] James T Rayfield, Shuxin Lin, Nianjun Zhou, and Dhaval C Patel. 2025. ReAct Meets Industrial IoT: Language Agents for Data Access. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, Saloni Potdar, Lina Rojas-Barahona, and Sebastien Montella (Eds.). Association for Computational Linguistics, Suzhou (China), 364–382. doi:10.18653/v1/2025.emnlp-industry.24
- [39] Philipp Schmid. 2025. *Pass@k* vs *Pass^k*: Understanding Agent Reliability. <https://www.philpschmid.de/agents-pass-at-k-pass-power-k> Accessed: 2026-01-01.
- [40] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=yHdTscY6Ci>
- [41] Yongliang Shen, Kaitao Song, Xu Tan, Wenqi Zhang, Kan Ren, Siyu Yuan, Weiming Lu, Dongsheng Li, and Yueting Zhuang. 2024. TaskBench: Benchmarking Large Language Models for Task Automation. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 4540–4574. https://proceedings.neurips.cc/paper_files/paper/2024/file/085185ea97db31ae6dcac7497616fd3e-Paper-Datasets_and_Benchmarks_Track.pdf
- [42] Shohei Shimizu, Takanori Inazumi, Yasuhiro Sogawa, Aapo Hyvarinen, Yoshinobu Kawahara, Takashi Washio, Patrik O Hoyer, Kenneth Bollen, and Patrik Hoyer. 2011. DirectLiNGAM: A direct method for learning a linear non-Gaussian structural equation model. *Journal of Machine Learning Research-JMLR* 12, Apr (2011), 1225–1248.
- [43] SkyFoundry contributors. 2026. SkySpark (software). <https://skyfoundry.com/>. Proprietary software; Accessed: 2026-01-01.
- [44] Yifan Song, Weimin Xiong, Xiutian Zhao, Dawei Zhu, Wenhao Wu, Ke Wang, Cheng Li, Wei Peng, and Sujian Li. 2024. AgentBank: Towards Generalized LLM Agents via Fine-Tuning on 50000+ Interaction Trajectories. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 2124–2141. doi:10.18653/v1/2024.findings-emnlp.116
- [45] Andrew Szot, Bogdan Mazouze, Omar Attia, Aleksei Timofeev, Harsh Agrawal, Devon Hjelm, Zhe Gan, Zsolt Kira, and Alexander Toshev. 2025. From multimodal llms to generalist embodied agents: Methods and lessons. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 10644–10655.
- [46] LangChain Team. 2025. Benchmarking Single Agent Performance. LangChain Blog. <https://blog.langchain.com/react-agent-benchmarking/> Accessed: 2026-01-01.
- [47] Bruno Veloso, Rita P Ribeiro, João Gama, and Pedro Mota Pereira. 2022. The MetroPT dataset for predictive maintenance. *Scientific Data* 9, 1 (2022), 764.
- [48] Lu Wang, Fangkai Yang, Chaoyun Zhang, Junting Lu, Jiaxu Qian, Shilin He, Pu Zhao, Bo Qiao, He Huang, Si Qin, Qisheng Su, Jiayi Ye, Yudi Zhang, Jianguang Lou, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2025. Large Action Models: From Inception to Implementation. *Transactions on Machine Learning Research* (2025). <https://openreview.net/forum?id=bYdKtfQ31>
- [49] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML'24)*. JMLR.org, Article 2054, 25 pages.
- [50] Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankut Shah, Yujia Bao, and Eugene Siow. 2025. MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers. arXiv:2508.20453 [cs.CL] <https://arxiv.org/abs/2508.20453>
- [51] Bosi Wen, Pei Ke, Xiaotao Gu, Lindong Wu, Hao Huang, Jinfeng Zhou, Wenchuang Li, Binxin Hu, Wendy Gao, Jiaxin Xu, Yiming Liu, Jie Tang, Hongning Wang, and Minlie Huang. 2024. Benchmarking Complex Instruction-Following with Multiple Constraints Composition. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 137610–137645. https://proceedings.neurips.cc/paper_files/paper/2024/file/f8c24b08b96a08ec7a7a975fee7777e-Paper-Datasets_and_Benchmarks_Track.pdf
- [52] Wikipedia contributors. 2025. Energy management system. [https://en.wikipedia.org/wiki/Energy_management_system_\(building_management\)](https://en.wikipedia.org/wiki/Energy_management_system_(building_management)). Accessed: 2025-11-25.
- [53] Wikipedia contributors. 2025. Enterprise asset management. https://en.wikipedia.org/wiki/Enterprise_asset_management. Accessed: 2025-11-25.
- [54] Wikipedia contributors. 2025. SCADA. <https://en.wikipedia.org/wiki/SCADA>. Accessed: 2025-11-25.
- [55] Yu Xia, Jingru Fan, Weize Chen, Siyu Yan, Xin Cong, Zhong Zhang, Yaxi Lu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2025. AgentRM: Enhancing Agent Generalization with Reward Modeling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 19277–19290. doi:10.18653/v1/2025.acl-long.945
- [56] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, and Dongkuan Xu. 2024. Decoupling Reasoning from Observations for Efficient Augmented Language Models. <https://openreview.net/forum?id=CpgoO6j6W1>
- [57] Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Zhiruo Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Melroy Maben, Raj Mehta, Wayne Chi, Lawrence Keunho Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. 2025. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. <https://openreview.net/forum?id=LZnKNpvH>
- [58] Zhen Xu, Sergio Escalera, Adrien Pavão, Magali Richard, Wei-Wei Tu, Quanming Yao, Huan Zhao, and Isabelle Guyon. 2022. Codabench: Flexible, easy-to-use, and reproducible meta-benchmark platform. *Patterns* 3, 7 (2022), 100543. doi:10.1016/j.patter.2022.100543
- [59] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R Narasimhan. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=roNSXZpUDN>
- [60] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X
- [61] Jianguo Zhang, Thai Hoang, Ming Zhu, Xuxin Liu, Shiyu Wang, Tulika Awalgaonkar, Akshara Prabhakar, Haolin Chen, Weiran Yao, Zhiwei Liu, Juntao Tan, Juan Carlos Nieves, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. 2025. ActionStudio: A Lightweight Framework for Data and Training of Large Action Models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 21488–21502. doi:10.18653/v1/2025.emnlp-main.1090
- [62] Jianguo Zhang, Tian Lan, Ming Zhu, Xuxin Liu, Thai Quoc Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Akshara Prabhakar, Haolin Chen, Zhiwei Liu, Yihao Feng, Tulika Manoj Awalgaonkar, Rithesh R N, Zeyuan Chen, Ran Xu, Juan Carlos Nieves, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. 2025. xLAM: A Family of Large Action Models to Empower AI Agent Systems. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 11583–11597. <https://aclanthology.org/2025.naacl-long.578/>
- [63] Nianjun Zhou, Dhaval Patel, and Anamitra Bhattacharyya. 2026. Deployed AI Agents for Industrial Asset Management: CodeReAct Framework for Event Analysis and Work Order Automation. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 47 (March 2026), 40176–40184. doi:10.1609/aaai.v40i47.41453

A Appendix

A.1 Novelty of the Orchestration Paradigms

A central contribution of ASSETOPS BENCH is an in-depth benchmark study that compares two orchestration paradigms side by side under realistic industrial conditions. While we adapt well-established paradigms, namely ReAct [60] for Agent-As-Tool, and ReWoo [56] together with Agent-Oriented Planning [26] for Plan-Execute, our implementations are not naive compositions of these prior works. Table 6 summarizes five non-trivial systems-level innovations required to make these paradigms reliable in long-horizon, multi-agent industrial settings, emerging directly from the practical constraints of long-horizon industrial reasoning.

Table 6: Systems-level innovations required to adapt the two orchestration paradigms to long-horizon industrial reasoning.

Agent-As-Tool
Dual-output interface. Each sub-agent returns structured JSON (with optional file pointers) plus a natural-language message summarizing the result, e.g., “<i>found 25 observations; file_path contains a JSON array.</i>” This lets the meta-agent decide whether to load a file into context. Context-aware tooling. The meta-agent loads file content only when needed, via dedicated utility tools, to mitigate context bloat. Agent-Ask (recovery). A lightweight mechanism letting sub-agents request missing information from the parent agent; raised task completion from 59% to 66%.
Plan-Execute
Executor design. Neither ReWoo nor Agent-Oriented Planning specifies the executor in detail; we provide a dedicated design and discussion. Context-aware memory. The planner specifies how context flows across sub-agents; the executor controls what is passed forward, supporting four memory-passing modes.

A.2 Impact of In-Context Examples

Table 7 compares gpt-4.1 and granite-3-3-8b with and without in-context examples on a subset of single-agent benchmark tasks ($n = 65$). Consistent with our main findings, in-context examples are critical for enabling effective ReAct-style reasoning and coordination. Removing them causes a large performance drop for both models: gpt-4.1 decreases from an average of 80% (with context) to 33% (without), while granite-3-3-8b falls from 60% to 3%. We omit work-order and end-to-end tasks here because their baseline performance is already low.

A.3 Baseline using Single-Agent

Instead of using four domain-specific sub-agents and an orchestration agent, we build a tool-calling ReAct agent with a single prompt as a baseline, giving it access to tools and in-context examples from all agents. This increases problem complexity, as the agent must handle many tools and an expanded context. Running the default LLM, llama-4-maverick, on all 141 scenarios, the single-agent

Table 7: Impact of in-context examples on single-agent tasks ($n = 65$). TC: task completion; DRA: data retrieval accuracy; RV: result verification. Scores are percentages.

Model	Ex.	TC	DRA	RV
gpt-4.1	Yes	52	57	55
	No	22	21	24
granite-3-3-8b	Yes	40	44	41
	No	2	3	3

baseline achieves task completion of 26.95%, data retrieval accuracy of 34.04%, and generalized result verification of 28.37%. Under the Agent-As-Tool setup, the same model achieves roughly two-fold improvements (see Figure 5a).

A.4 Cost Analysis

Table 8 compares the estimated cost of running the full 140+ utterance task suite under the Agent-As-Tool paradigm, reporting the average tokens consumed per task and the resulting total cost for each provider/model configuration. Token usage is comparable across the two models, but the total estimated cost differs substantially, reflecting provider pricing rather than task-specific token demand: llama-4-maverick yields a lower total cost while maintaining similar average tokens per task.

Table 8: Runtime and estimated cost for executing 140+ utterance tasks using the Agent-As-Tool paradigm.

LLM	Provider	Avg. tokens	Total cost (USD)
gpt-4.1	OpenAI	3,664	\$300
llama-4-maverick	watsonx	3,730	\$130

A.5 Uncertainty Analysis

As discussed in Section 5.2, we run the evaluation agent five times to estimate the stability of rubric-based metrics. Table 9 reports inter-rater agreement across the five runs, together with the derived uncertainty, computed as $1 - \text{agreement}$. Agreement is consistently high (≥ 0.968), corresponding to uncertainties below 3.2%, indicating that the LLM-as-judge evaluation is stable under repeated runs.

Table 9: Inter-rater agreement and derived uncertainty across five evaluation runs.

Metric	Agreement	Uncertainty
Task Completion	0.9731	2.69%
Data Retrieval Accuracy	0.9697	3.03%
Generalized Result Verification	0.9681	3.19%

We have made the dataset and an extended technical report publicly available in our code repository, as referenced in the abstract.